WEJ LAB

Decision Trees and Random Forests

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-11-09

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Explain how a tree partitions the predictors with yes/no splits
- Fit and prune regression and classification trees (`rpart`, `caret`)
- Explain why single trees are unstable, and how bagging fixes it
- Explain the two sources of randomness in a random forest
- Tune a random forest, read its OOB error and variable importance

Road map

One idea, built up in four steps:

1. **Olive oil** (classification): a tree as a set of yes/no rules
2. **2008 polls** (regression, one predictor): how splits are chosen; overfitting and pruning
3. **Digits 2 vs 7** (classification, two predictors): a tree's blocky boundary
4. **Random forests**: average many trees; then back to **olive oil**

WEJ LAB
RUTGERS HEALTH
Center for Data Science

WEJ LAB

Decision trees

RUTGERS HEALTH
Center for Data Science

Motivating example: olive oil

Composition of Italian olive oils: 8 fatty acids, 3 regions.

```
data("olive")
olive <- select(olive, -area) # remove `area`; we predict `region`
names(olive)
```

```
## [1] "region"      "palmitic"    "palmitoleic" "stearic"     "oleic"
## [6] "linoleic"    "linolenic"   "arachidic"   "eicosenoic"
```

```
table(olive$region)
```

```
##
## Northern Italy       Sardinia Southern Italy
##            151             98            323
```

Motivating example: olive oil

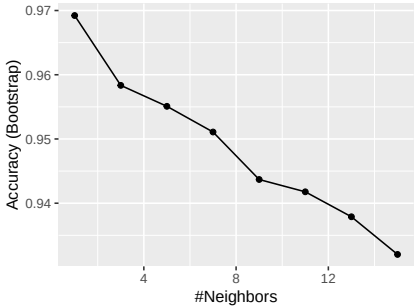
```
olive %>% head() %>%
  knitr::kable() %>% kable_styling(font_size = 7)
```

region	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eicosenoic
Southern Italy	10.75	0.75	2.26	78.23	6.72	0.36	0.60	0.29
Southern Italy	10.88	0.73	2.24	77.09	7.81	0.31	0.61	0.29
Southern Italy	9.11	0.54	2.46	81.13	5.49	0.31	0.63	0.29
Southern Italy	9.66	0.57	2.40	79.52	6.19	0.50	0.78	0.35
Southern Italy	10.51	0.67	2.59	77.71	6.72	0.50	0.80	0.46
Southern Italy	9.11	0.49	2.68	79.24	6.78	0.51	0.70	0.44

Motivating example: olive oil

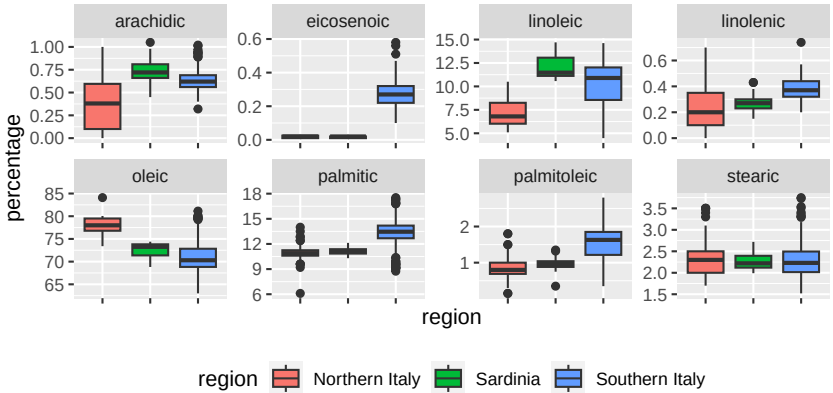
A quick kNN fit already predicts well:

```
fit <- train(region ~ ., method = "knn", data = olive,
             tuneGrid = data.frame(k = seq(1, 15, 2)))
ggplot(fit)
```



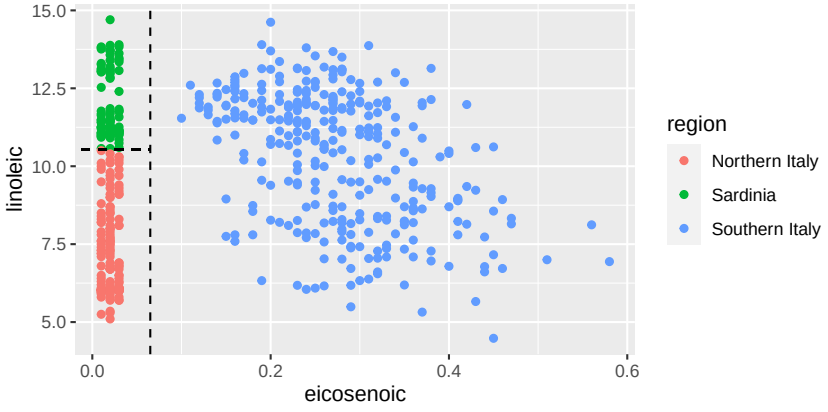
Motivating example: olive oil

Eicosenoic is only in Southern Italy; linoleic separates Northern Italy from Sardinia:



Motivating example: olive oil

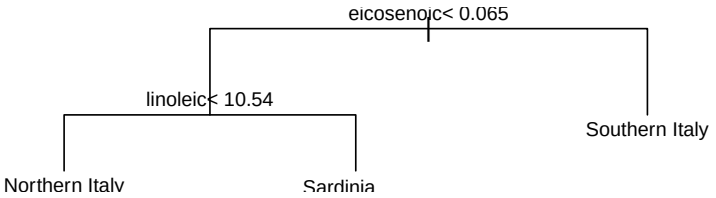
Two predictors should separate the regions almost perfectly:



WEJ LAB
RUTGERS HEALTH
Center for Data Science

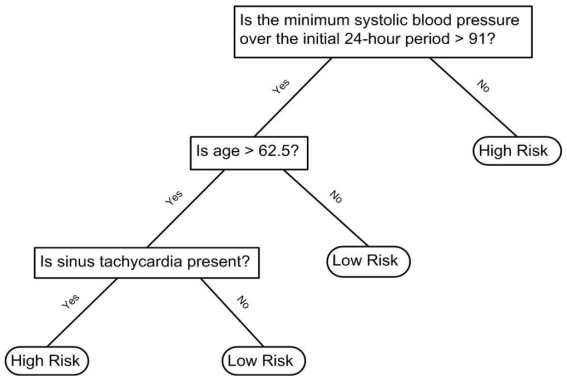
Motivating example: olive oil

Decision rule: eicosenoic $> 0.065 \rightarrow$ Southern Italy; otherwise linoleic $> 10.535 \rightarrow$ Sardinia, else Northern Italy.



Decision trees in medicine

Trees like this are common in clinical practice, e.g., risk after a heart attack:



(Source: Walton 2010 Informal Logic, Vol. 30, No. 2, pp. 159-184.)

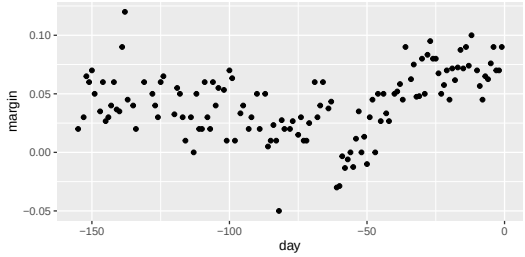
Decision trees

- A tree is a **flow chart of yes/no questions**, with predictions at the ends (**leaves**)
- **CART** (Classification and Regression Trees) predicts an outcome Y by **partitioning the predictors X**
- How are the splits chosen? Easiest to see with **one** predictor $\rightarrow$

Regression trees

Continuous outcome → **regression tree**. Estimate $f(x) = \mathrm{E}(Y|X = x)$: $Y =$ poll margin (Obama − McCain), $x =$ day.

```
data("polls_2008")
polls_2008 %>% ggplot() + geom_point(aes(day, margin))
```



Regression trees

- Partition the predictor space into J non-overlapping regions $R_1, \ldots, R_J$
- For any $x \in R_j$, predict the **average of the training** y_i in R_j
- Partitions are built **recursively**: start with one region, split it in two, then split one of those, ...

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Regression trees: choosing a split

At each step, search **every predictor** j **and cutpoint** s and pick the split $R_1 = \{x_j < s\}$, $R_2 = \{x_j \geq s\}$ that minimizes

$$\sum_{i:\, x_i \in R_1} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2} (y_i - \hat{y}_{R_2})^2$$

A **greedy** search: the best split *now*, not the best tree overall.

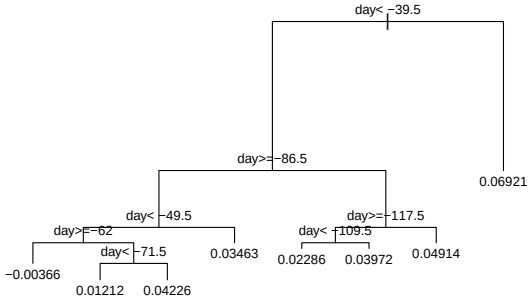
Regression trees

Fit with the `rpart` function in the **rpart** package:

```
fit <- rpart(margin ~ ., data = polls_2008)
```

Regression trees

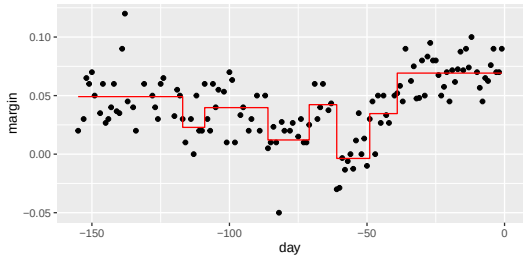
The splits:



Regression trees

First split at day -39.5, then -86.5, and so on. The estimate $\hat{f}(x)$ is a step function:

```
polls_2008 %>%
  mutate(y_hat = predict(fit)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col = "red")
```



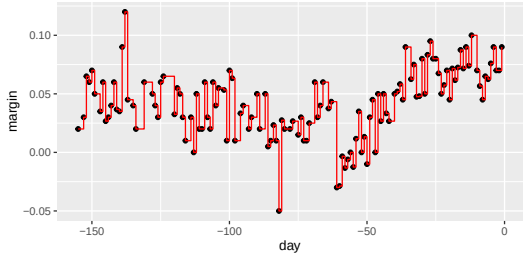
Regression trees: when to stop?

- Each split **lowers training RSS**
- Split until every point is its own region $\rightarrow$ RSS $= 0$ (**overfitting**)
- `rpart` stops a split unless it improves fit by the **complexity parameter** `cp`
- Also: **`minsplit`** (min. observations to split a node, default 20) and **`minbucket`** (min. per leaf)

Regression trees

`cp = 0` and `minsplit = 2`: the tree just reproduces the data:

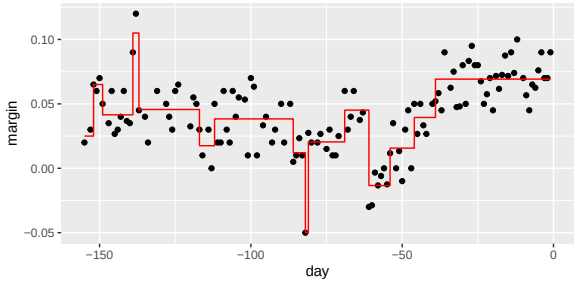
```
fit <- rpart(margin ~ ., data = polls_2008,
             control = rpart.control(cp = 0, minsplit = 2))
polls_2008 %>%
  mutate(y_hat = predict(fit)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col = "red")
```



Regression trees

Pruning cuts back an existing tree to a larger cp:

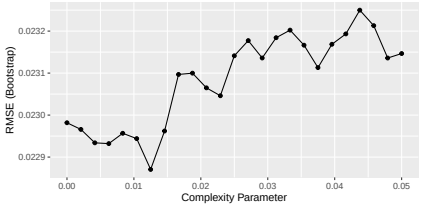
```
pruned_fit <- prune(fit, cp = 0.01)
polls_2008 %>% mutate(y_hat = predict(pruned_fit)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col = "red")
```



Regression trees

How to pick cp? **Cross-validation:**

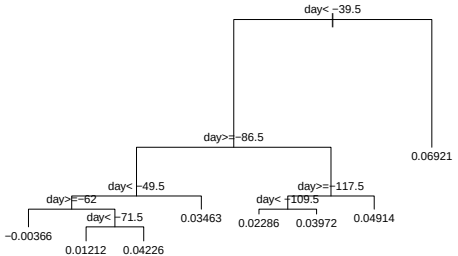
```
train_rpart <- train(margin ~ ., method = "rpart",
                     tuneGrid = data.frame(cp = seq(0, 0.05, len = 25)),
                     data = polls_2008)
ggplot(train_rpart)
```



Regression trees

The final tree (`finalModel`):

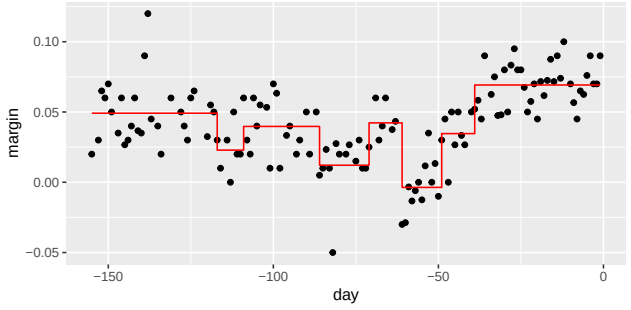
```
plot(train_rpart$finalModel, margin = 0.1)
text(train_rpart$finalModel, cex = 0.75)
```



Regression trees

With one predictor we can plot $\hat{f}(x)$:

```
polls_2008 %>%
  mutate(y_hat = predict(train_rpart)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col = "red")
```



Classification (decision) trees

Used when the **outcome is categorical**. Two changes from regression trees:

- **Predict** the *most common class* in each partition (not the average)
- **Split** to make partitions **pure**, using the **Gini index** or **entropy** (instead of RSS)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Classification (decision) trees

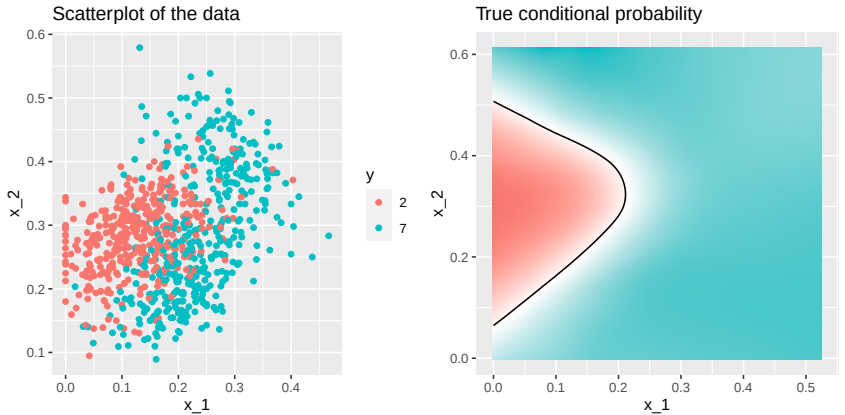
Let $\hat{p}_{j,k}$ be the proportion of class k in partition j:

$$\text{Gini}(j) = \sum_{k=1}^{K} \hat{p}_{j,k}(1 - \hat{p}_{j,k}) \qquad \text{entropy}(j) = -\sum_{k=1}^{K} \hat{p}_{j,k} \log(\hat{p}_{j,k})$$

- Both are 0 when a partition is pure (one class); $0 \times \log(0)$ is defined as 0
- Both grow as the classes get more mixed

Classification (decision) trees

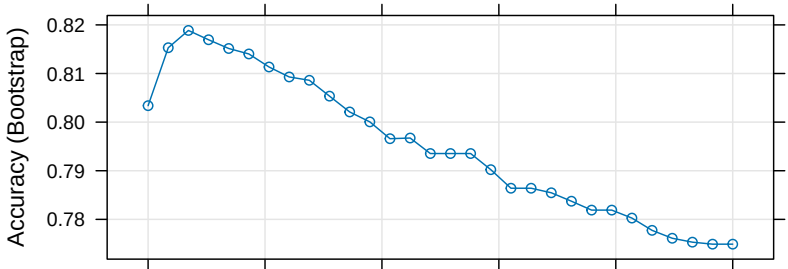
Digits 2 vs. 7, predicted from two pixel summaries (x_1, x_2):



Classification (decision) trees

Tuning the complexity parameter:

```
set.seed(1)
train_rpart <- train(y ~ ., method = "rpart", data = mnist_27$train,
                     tuneGrid = data.frame(cp = seq(0.0, 0.1, len = 30)))
plot(train_rpart)
```



WEJ LAB
RUTGERS HEALTH
Center for Data Science

Classification (decision) trees

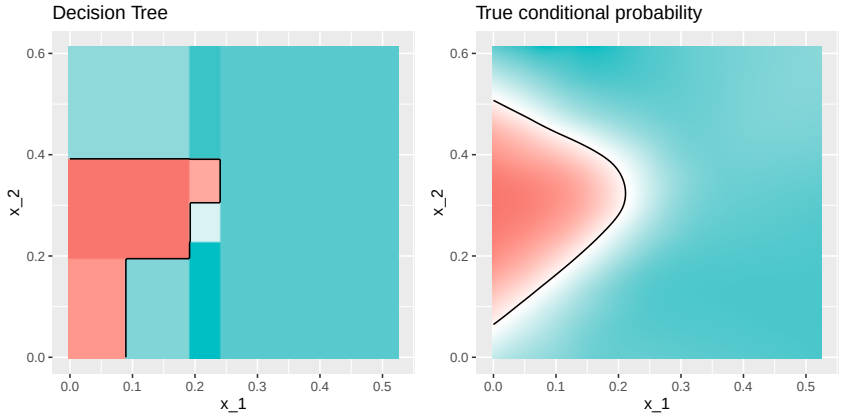
Test-set accuracy:

```
y_hat <- predict(train_rpart, mnist_27$test)
confusionMatrix(y_hat, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.82
```

Classification (decision) trees

The estimated boundary is **blocky**: every split is parallel to an axis.



Trees: strengths and weaknesses

Strengths:

- Highly interpretable; easy to draw (if small)
- Mimic human decision rules; no dummy variables needed

Weaknesses:

- Easy to overfit; rarely the most accurate method
- **High variance:** small changes in the data can change the first split, and so the whole tree

WEJ LAB

Random forests

RUTGERS HEALTH
Center for Data Science

Random forests

Random forests fix the instability of single trees by **averaging many trees** (Breiman 2001):

1. **Build** B trees $T_1, \ldots, T_B$, each made randomly different
2. **Predict** each case with every tree: $\hat{y}_b$ from tree T_b
3. **Combine:** average $\hat{y} = \frac{1}{B} \sum_{b=1}^{B} \hat{y}_b$ (regression) or **majority vote** (classification)

Random forests: two sources of randomness

1. **Bootstrap the rows:** each tree is fit to N observations sampled *with replacement* (**bagging** = bootstrap aggregation)
2. **Sample the predictors:** at **each split**, only a random subset of `mtry` predictors is considered

Step 2 **decorrelates** the trees (one strong predictor can't dominate every tree), so averaging reduces variance more.

Random forests

Polls data, **randomForest** package. (One predictor, so this forest is just **bagging**.)

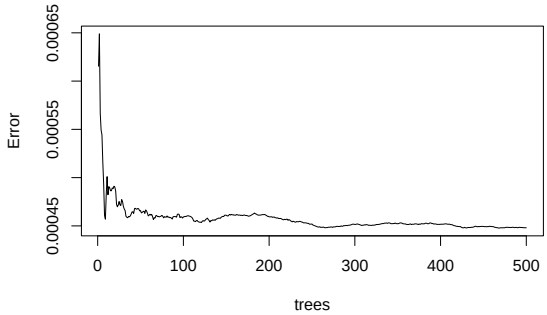
```
fit <- randomForest(margin ~ ., data = polls_2008)
fit
```

```
##
## Call:
##  randomForest(formula = margin ~ ., data = polls_2008)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 1
##
##           Mean of squared residuals: 0.0004480303
##                     % Var explained: 46.32
```

Random forests

Error vs. number of trees: it levels off after about 30 trees.

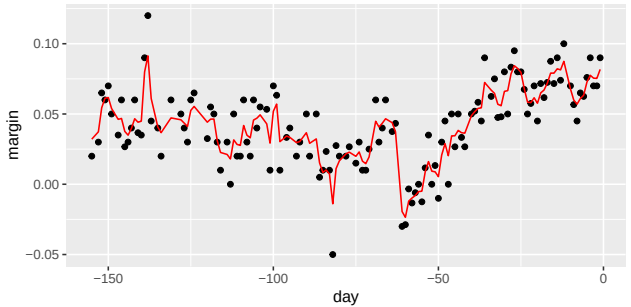
```
plot(fit, main = '')
```



Random forests

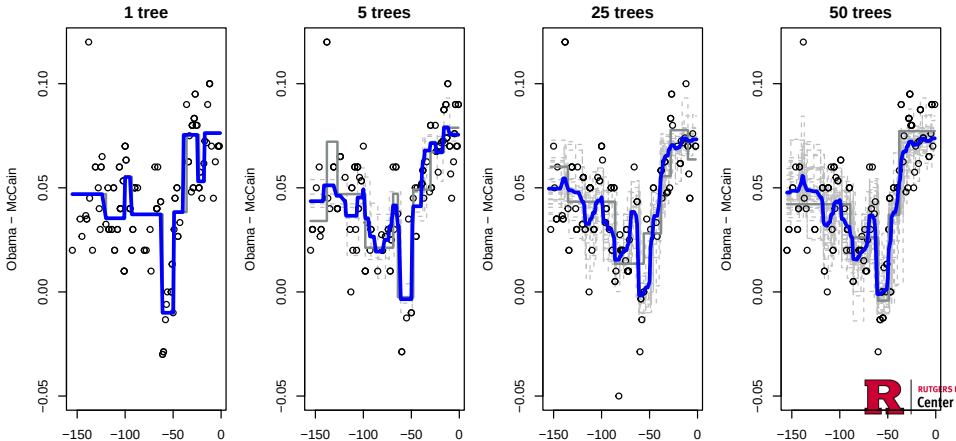
The forest's estimate is much smoother than a single tree:

```
polls_2008 %>%
  mutate(y_hat = predict(fit, newdata = polls_2008)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_line(aes(day, y_hat), col = "red")
```



Random forests: averaging bootstrap trees

Grey: trees from individual bootstrap samples. Blue: their average.



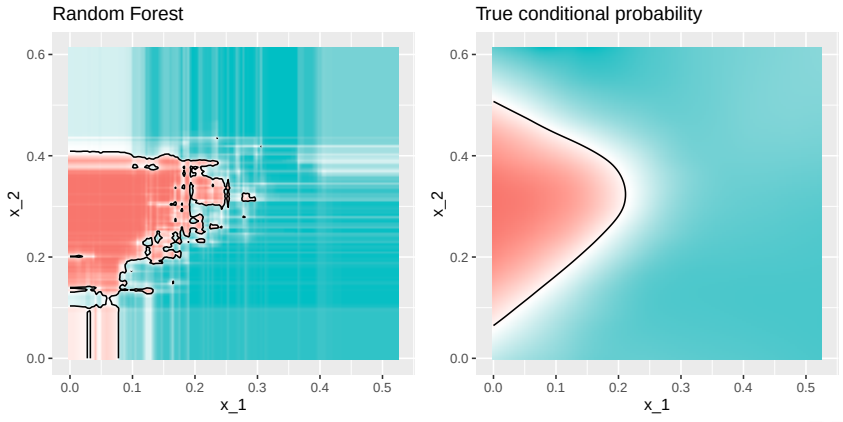
Random forests: digits

Back to digits 2 vs. 7:

```
set.seed(1)
train_rf <- randomForest(y ~ ., data = mnist_27$train)
confusionMatrix(predict(train_rf, mnist_27$test),
                mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.79
```

Random forests: digits

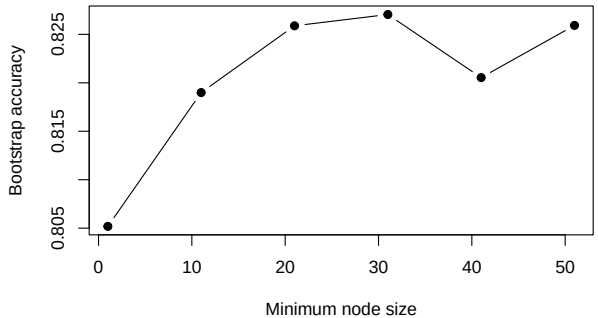


Random forests: tuning node size

More accurate, but the estimate is noisy. A larger **minimum node size** (points per leaf) gives a smoother estimate; tune it with **caret**:

```
set.seed(1)
nodesize <- seq(1, 51, 10)
acc <- sapply(nodesize, function(ns){
  train(y ~ ., method = "rf", data = mnist_27$train,
        tuneGrid = data.frame(mtry = 2),
        nodesize = ns)$results$Accuracy })
plot(nodesize, acc)
```

Random forests: tuning node size



Random forests: tuning node size

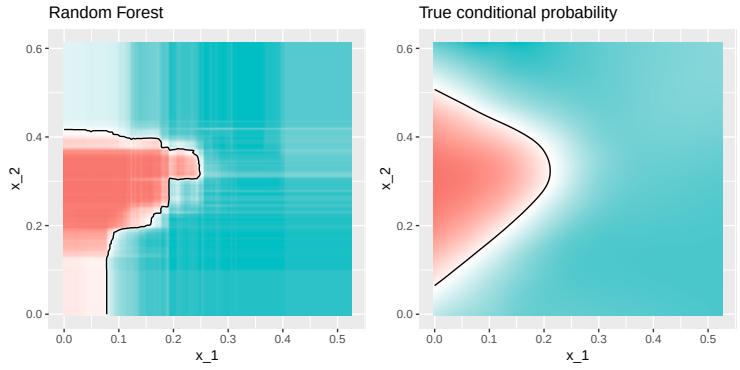
Refit with the best node size and evaluate on the test set:

```
set.seed(1)
train_rf_2 <- randomForest(y ~ ., data = mnist_27$train,
                           nodesize = nodesize[which.max(acc)])
confusionMatrix(predict(train_rf_2, mnist_27$test),
                mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.83
```

Random forests: tuning node size

A smoother estimate, closer to the truth:



Out-of-bag (OOB) error: validation for free

- Each bootstrap sample leaves out $\approx 1/3$ of the cases (**out-of-bag**)
- Predict each case using only the trees that **never saw it**
- Average over cases $\rightarrow$ an **honest error estimate** without a separate validation set
- `randomForest()` prints it as the "OOB estimate of error rate"

Back to olive oil

A random forest on all 8 fatty acids (`importance = TRUE` for permutation importance):

```
set.seed(1)
olive_rf <- randomForest(region ~ ., data = olive, importance = TRUE)
olive_rf$confusion   # OOB confusion matrix
```

```
##                 Northern Italy Sardinia Southern Italy class.error
## Northern Italy             151        0              0           0
## Sardinia                     0       98              0           0
## Southern Italy               0        0            323           0
```

OOB error rate: 0%.

Variable importance

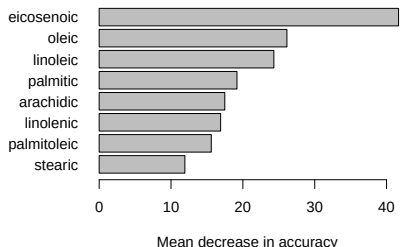
Forests lose the tree's interpretability. **Variable importance** recovers some of it:

- **Permutation importance** (`MeanDecreaseAccuracy`): shuffle one predictor in the OOB data; how much does accuracy drop?
- **Impurity importance** (`MeanDecreaseGini`): total decrease in Gini from splits on that predictor, summed over all trees
- `caret::varImp()` extracts importance from any supported model

Variable importance: olive oil

Eicosenoic tops the list. Oleic ranks with linoleic: the two are strongly correlated ($r \approx -0.85$), so they share the credit.

```
imp <- importance(olive_rf, type = 1)  # permutation importance
par(mar = c(4, 7, 1, 1))
barplot(sort(imp[, 1]), horiz = TRUE, las = 1,
        xlab = "Mean decrease in accuracy")
```



Variable importance: cautions

- **Correlated predictors share importance:** two co-expressed genes split the credit, so each can look weak
- **Impurity importance is biased** toward continuous and many-level predictors (more possible cutpoints); permutation importance is less so
- Importance is **predictive, not causal**
- More on explaining models (SHAP, LIME) next week

Trees & forests in practice

When are random forests a good choice?

- **Non-linear** effects and **interactions** among predictors
- **Mixed** predictor types, minimal preprocessing (no scaling needed)
- **Many predictors**, even $p \gg N$ (e.g., genes)
- Strong performance with **little tuning**

Trees & forests in practice

Knobs (and sensible defaults):

- `ntree`: enough trees for the error curve to flatten (often 500)
- `mtry`: predictors tried per split; $\approx \sqrt{p}$ (classification), $p/3$ (regression)
- `nodesize`: larger $\rightarrow$ smoother, less overfitting

Tune `mtry` and node size by **cross-validation**. `ranger` is a fast implementation (`method = "ranger"` in caret).

Trees & forests in practice

Omics / high-dimensional data ($p \gg N$):

- Forests handle **thousands of genes** with few samples
- With many noise genes, a small `mtry` can miss the signal: **tune it**
- **Class imbalance** (e.g., rare disease): use class weights, balanced sampling, or a tuned cutoff
- **Never judge performance on training data**: use OOB, cross-validation, or a held-out test set

WEJ LAB

Summary

- A tree partitions the predictors with greedy yes/no splits (RSS for regression, Gini/entropy for classification)
- Trees overfit unless stopped or pruned (`cp`, `minsplit`); choose `cp` by cross-validation
- Single trees are interpretable but unstable (high variance)
- Random forests average bootstrap trees and sample `mtry` predictors at each split
- OOB error gives honest validation for free
- Variable importance helps interpret forests, but beware correlated predictors

Python companion

Python versions of these examples (`scikit-learn`'s `DecisionTreeRegressor`, `DecisionTreeClassifier`, `RandomForestClassifier`):

`python/week6/trees.ipynb`

WEJ LAB

RUTGERS HEALTH
Center for Data Science

Acknowledgments

Examples, figures, and several explanations are adapted from R. A. Irizarry, *Introduction to Data Science* (the `dslabs` book) — licensed CC BY-NC-SA.

This deck was edited by Claude AI.

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] gridExtra_2.3        kableExtra_1.4.0     randomForest_4.7-1.1
##  [4] rpart_4.1.23         dslabs_0.7.6         lubridate_1.9.3
##  [7] forcats_1.0.0        stringr_1.5.1        dplyr_1.1.4
## [10] purrr_1.0.2          readr_2.1.5          tidyr_1.3.1
## [13] tibble_3.2.1         tidyverse_2.0.0      caret_6.0-94
## [16] lattice_0.22-5       ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
```